



Storitveno usmerjene arhitekture

izr. prof. dr. Boštjan Šumak

1

Nazadnje

- Strateški cilji SOA,
- SOA Manifesto,
- Vzorci SOA,
- Aritekturni stil REST,
- Storitve REST,
- Vzorci in antivzorci REST,
- WS* vs REST,
- SOA z REST (ROA)

2

Razvoj storitev REST

3

Java API for RESTful Web Services (JAX-RS)

- Standardni API za razvoj REST-spletnih storitev v programskem jeziku Java
- Prvotno del Java EE, danes del Jakarta EE
- Namen:
 - poenostaviti razvoj javanskih spletnih storitev,
 - ki sledijo arhitekturnemu slogu REST
- Temelji na anotacijah, ki jih zapisujemo neposredno v izvorni kodi:
 - opisujejo vire,
 - določajo URI-je in HTTP metode,
 - definirajo formate vsebine.

4

JAX-RS API zagotavlja

- Anotacije za vire in komponente aplikacije
 - označevanje razredov in metod kot vire (**@Path**, **@GET**, ...)
- Povezavo URI-jev in HTTP metod z metodami razredov
 - vzorci poti (npr. **/racuni/{id}**) se preslikajo na proženje konkretnih metod
- Vezavo parametrov iz HTTP zahteve na parametre metode
 - pot (**@PathParam**), poizvedba (**@QueryParam**), zaglavja (**@HeaderParam**), obrazci (**@FormParam**), ...
- Delo s telesom sporočila
 - bralci in zapisovalci (**MessageBodyReader** / **MessageBodyWriter**)
 - ni nam treba ročno skrbeti za (de)serializacijo v **JSON/XML**
- Samodejno pretvorbo v XML in JSON
 - z uporabo knjižnic, kot so **JAXB**, **Jackson**, ...
- Preslikavo izjem v HTTP odzive - **ExceptionHandler** omogoča lastne napake in kode (**4xx**, **5xx**)
- Pogajanje o formatu vsebine (content negotiation)
 - podpora zaglavjem **Accept**, **Content-Type** ...
- API za odjemalce
 - **Client API** za klicanje REST storitev iz Jave

5

Pogosto uporabljene JAX-RS anotacije

- **@ApplicationPath** - osnovna pot aplikacije (koren vseh virov)
- **@Path** - pot do vira ali metode (npr. **/api/racuni**)
- **@GET**, **@POST**, **@PUT**, **@DELETE**, **@PATCH**, **@HEAD**, **@OPTIONS** - preslikava HTTP metod
- **@Produces** - kateri medijski tipi so v odgovoru (npr. **application/json**)
- **@Consumes** - kateri medijski tipi se sprejemajo v zahtevi
- **@PathParam** - vrednost iz poti URI (**/racuni/{id}**)
- **@QueryParam** - vrednost iz poizvedbe (**?page=2**)
- **@HeaderParam**, **@CookieParam**, **@FormParam** - vrednosti iz zaglavij, piškotkov, obrazcev
- in številne druge (npr. **@Context** za dostop do **UriInfo**, **Request**, ...)

6

Implementacije JAX-RS

Jersey – referenčna
odprtokodna
implementacija
(Oracle / Eclipse)

RESTEasy –
implementacija
skupine JBoss /
Red Hat

Apache CXF –
ogrodje za SOAP in
REST storitve

RESTlet – ogrodje
za spletne storitve
in aplikacije (REST)

Apache Wink in
druge manj
razširjene
implementacije

V praksi pride JAX-RS pogosto že **vključen v aplikacijski strežnik**
(npr. WildFly, Payara, TomEE ...), lahko pa ga uporabimo tudi v samostojnih javanskih aplikacijah.

7

Jersey

- Odprtokodna implementacija JAX-RS
- Lahko deluje kot:
 - aplikacija v vsebniki servletov (Tomcat, Jetty, ...)
 - vgrajeni HTTP strežnik (npr. Jersey + Grizzly v main())
- Ponuja:
 - samodejno generiranje WADL (opis storitve),
 - podporo za različne medijske tipe (JSON, XML, Atom, večdelni MIME ...),
 - razširitve za varnost, filtriranje zahtev, logiranje ...
- V številnih aplikacijskih strežnikih je Jersey (ali druga implementacija JAX-RS) že integriran, zato ga ni treba posebej dodajati.

8

RESTEasy

- Implementacija JAX-RS iz okolja JBoss / Red Hat
- Namenjena je:
 - uporabi v aplikacijskih strežnikih (npr. WildFly),
 - ali kot knjižnica v samostojnih aplikacijah.
- Značilnosti:
 - tesna integracija z ekosistemom JBoss/WildFly,
 - podpora dodatnim razširitvam (varnost, validacija, interceptors ...),
 - možnost embediranja v različne servlet vsebnike.
- V strežnikih WildFly in nekaterih drugih je RESTEasy privzeta JAX-RS implementacija.

9

Primer: storitev racuni-rest na osnovi JAX-RS + Jersey

- Storitev z naslednjim vmesnikom:

API	Opis	Telo zahteve	Telo odgovora
GET /api/racuni	Vrne seznam vseh računov	/	polje objektov racun (XML ali JSON)
GET /api/racuni/{id}	Vrne račun z vrednostjo id	/	objekt Racun (XML ali JSON)
POST /api/racuni	Ustvari nov vir račun	objekt Racun	objekt Racun (XML ali JSON)
PUT /api/racuni/{id}	Posodobi obstoječ vir račun z vrednostjo ISBN	objekt Racun	/
DELETE /api/racuni/{id}	Zbriše vir račun z vrednostjo ISBN	/	/

10

Ustvarjanje projekta

**mvn archetype:generate **

**-DgroupId=si.um.feri **

**-DartifactId=racuni-rest **

**-DarchetypeArtifactId=maven-archetype-quickstart **

-DinteractiveMode=false

11

Pom.xml (jersey + glassfish)

```
<dependencies>
  <dependency>
    <groupId>jakarta.ws.rs</groupId>
    <artifactId>jakarta.ws.rs-api</artifactId>
    <version>4.0.0</version>
  </dependency>

  <dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
    <artifactId>jersey-container-grizzly2-http</artifactId>
    <version>${jersey.version}</version>
  </dependency>

  <dependency>
    <groupId>org.glassfish.jersey.inject</groupId>
    <artifactId>jersey-hk2</artifactId>
    <version>${jersey.version}</version>
  </dependency>

  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-json-jackson</artifactId>
    <version>${jersey.version}</version>
  </dependency>
</dependencies>
```

12

Racun.java

```
package si.um.feri.racuni;

public class Racun {

    private String id;
    private String datum;
    private String kraj;
    private double znesek;
    private String ppId;
    private String davcnaSt;

    public Racun() {}

    public Racun(String id, String datum, String kraj, double znesek, String ppId, String davcnaSt) {
        this.id = id;
        this.datum = datum;
        this.kraj = kraj;
        this.znesek = znesek;
        this.ppId = ppId;
        this.davcnaSt = davcnaSt;
    }

    + GETTER in SETTER METODE
}
```

13

Main.java

```
package si.um.feri.racuni;
import org.glassfish.grizzly.http.server.HttpServer;
import org.glassfish.jersey.grizzly2.httpserver.GrizzlyHttpServerFactory;
import org.glassfish.jersey.server.ResourceConfig;
import java.io.IOException;
import java.net.URI;

public class Main {

    public static final String BASE_URI = "http://localhost:8080/";

    public static HttpServer startServer() {
        // ResourceConfig registrira vse JAX-RS vire v paketu
        final ResourceConfig rc = new ResourceConfig().packages("si.um.feri.racuni");
        return GrizzlyHttpServerFactory.createHttpServer(URI.create(BASE_URI), rc);
    }

    public static void main(String[] args) throws IOException {
        final HttpServer server = startServer();
        System.out.println("Strežnik teče na " + BASE_URI + "api/racuni");
        System.out.println("Pritisni Enter za ustavitev...");
        System.in.read();
        server.shutdownNow();
    }
}
```

14

RacuniResource.java

```
package si.um.feri.racuni;
import jakarta.ws.rs.*;
import jakarta.ws.rs.core.*;
import java.net.URI;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
import java.util.stream.Collectors;
import java.util.UUID;

@Path("/api/racuni")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class RacuniResource {

    private static final Map<String, Racun> DATA = new ConcurrentHashMap<>();

    @GET
    @POST
    @PUT
    @DELETE
}
```

15

RacuniResource.java - **GET**

```
@GET
public Collection<Racun> getVsiRacuni() {
    return DATA.values();
}

@GET
@Path("/{id}")
public Response getRacun(@PathParam("id") String id) {

    Racun r = DATA.get(id);
    if (r == null) {
        return Response.status(Response.Status.NOT_FOUND).build();
    }
    return Response.ok(r).build();
}
```

16

RacuniResource.java - **POST**

```
@POST
public Response ustvariRacun(Racun racun, @Context UriInfo uriInfo) {

    String id = UUID.randomUUID().toString();
    racun.setId(id);
    DATA.put(id, racun);

    URI location = uriInfo.getAbsolutePathBuilder()
        .path(id)
        .build();

    return Response.created(location).entity(racun).build();
}
```

17

RacuniResource.java - **PUT**

```
@PUT
@Path("/{id}")
public Response updateRacun(@PathParam("id") String id, Racun racun){

    if (!id.equals(racun.getId()))
        return Response.status(Response.Status.BAD_REQUEST)
            .entity("Idja nista enaka!")
            .build();

    Racun najdi = DATA.get(id);

    if (najdi==null)
        return Response.status(Response.Status.NOT_FOUND)
            .entity("Računa z id = " + id + "ni v bazi")
            .build();

    DATA.put(id, racun);
    return Response.noContent().build();
}
```

18

RacuniResource.java - **DELETE**

```
@DELETE
@Path("/{id}")
public Response izbrisiRacun(@PathParam("id") String id) {

    Racun odstranjeni = DATA.remove(id);

    if (odstranjeni==null)
        return Response.status(Response.Status.NOT_FOUND)
            .entity("Računa z id = " + id + "ni v bazi")
            .build();

    return Response.noContent().build();
}
```

19

**REST storitev s CRUD
operacijami v Javi na
osnovi ogrodja Spring Boot**

20

Ogrodje



- Java ogrodje za razvoj spletnih aplikacij in spletnih API-jev, zgrajeno na ogrodju Spring
- Namenjeno je predvsem:
 - razvoju REST storitev,
 - klasičnih spletnih aplikacij,
 - mikrostoritev.
- Zaradi
 - velike skupnosti,
 - bogate dokumentacije in primerov,
 - dobre podpore v orodjih
- je Spring Boot eno najbolj razširjenih ogrodij za razvoj spletnih aplikacij in API-jev v Javi.

21

Glavne značilnosti



- **Minimalna konfiguracija**
 - omogoča razvoj REST API-jev in spletnih aplikacij z zelo malo ročne konfiguracije
 - veliko nastavitvev je vnaprej pripravljenih (»convention over configuration«).
- **Vgrajeno izvajalno okolje**
 - aplikacije se zaganjajo kot samostojni programi (java -jar),
 - z vgrajenim strežnikom (Tomcat, Jetty ali Undertow),
 - brez potrebe po zunaj nameščenem aplikacijskem strežniku.
- **Samodejna konfiguracija in upravljanje odvisnosti**
 - Spring Boot vsebuje avto-konfiguracijo za številne Spring module in zunanje knjižnice (JPA, Hibernate, Security, Messaging ...),
 - preko t. i. starter odvisnosti poenostavi dodajanje in upravljanje knjižnic.
- **Poenostavljen razvoj**
 - omogoča hitro prototipiranje,
 - razvijalcem prihrani veliko infrastrukturnega dela (nastavitve strežnika, povezave na bazo, logging ...),
 - osredotoči se lahko na poslovno logiko.

22

Glavne značilnosti



- **Prilagodljivost in razširljivost** - kljub samodejni konfiguraciji ima razvijalec vedno možnost:
 - ročno prilagoditi konfiguracijo,
 - zamenjati privzete komponente,
 - dopolniti delovanje glede na specifične zahteve.
- **Podpora za testiranje** - integrirana podpora za:
 - teste enot,
 - integracijske teste,
 - testiranje spletnih vmesnikov (MockMvc, TestRestTemplate ...).
- **Ogrodje, primerno za mikrororitve**
 - zaradi samostojnega zagona in lahke konfiguracije je ogrodje Spring Boot zelo primerno za mikrororitveno arhitekturo,
 - pogosto se uporablja skupaj z Spring Cloud in podobnimi orodji za gradnjo porazdeljenih sistemov.

23

Primer: storitev racuni-rest na osnovi spring boot

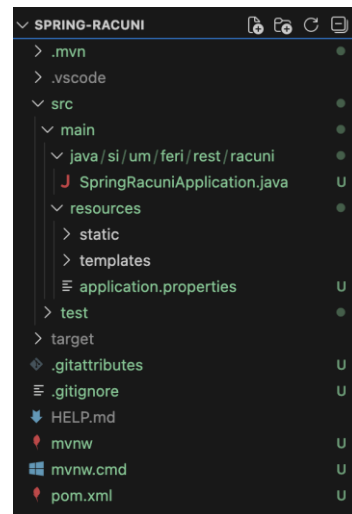
- Storitev z naslednjim vmesnikom:

API	Opis	Telo zahteve	Telo odgovora
GET /api/racuni	Vrne seznam vseh računov	/	polje objektov racun (XML ali JSON)
GET /api/racuni/{id}	Vrne račun z vrednostjo id	/	objekt Racun (XML ali JSON)
POST /api/racuni	Ustvari nov vir račun	objekt Racun	objekt Racun (XML ali JSON)
PUT /api/racuni/{id}	Posodobi obstoječ vir račun z vrednostjo ISBN	objekt Racun	/
DELETE /api/racuni/{id}	Zbriše vir račun z vrednostjo ISBN	/	/

24

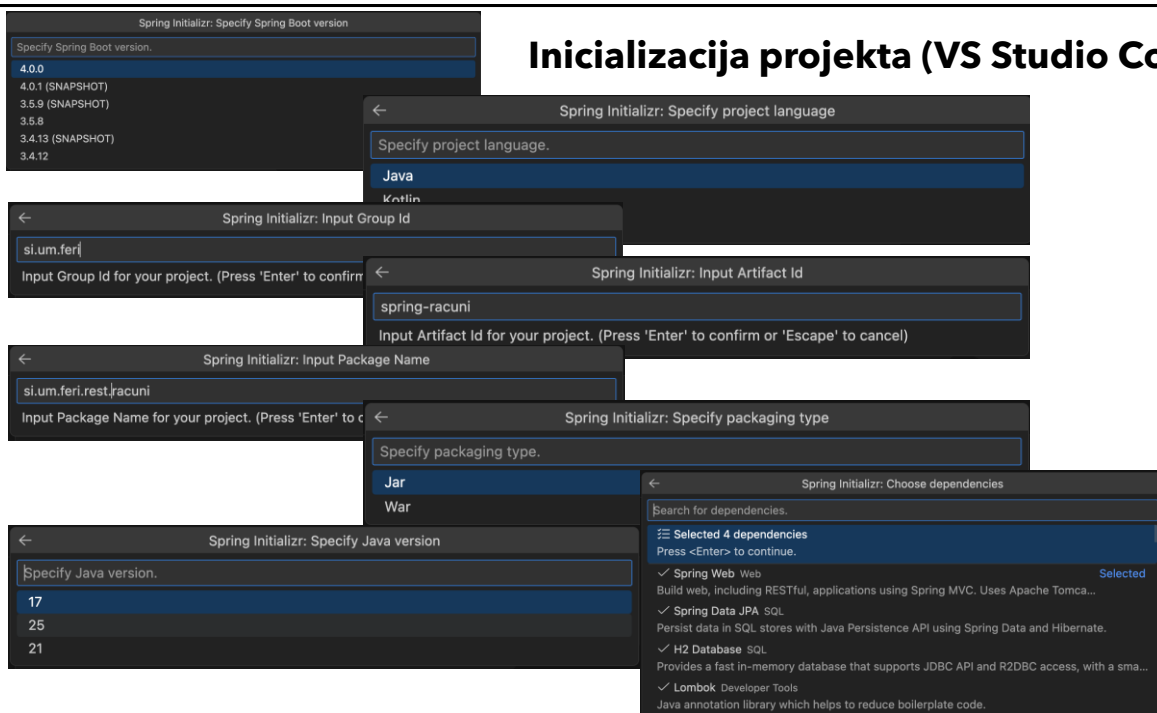
Inicializacija novega projekta

- Ustvarimo nov Spring Boot projekt in vključimo naslednje knjižnice:
 - **Spring Web**
 - **Spring Data JPA SQL**
 - **H2 Database SQL**
 - **Lombok DEVELOPER TOOLS**



25

Inicializacija projekta (VS Studio Code)



26

application.properties - Dodamo konfiguracijo:

```
spring.application.name=spring-racuni

# H2 baza v pomnilniku
spring.datasource.url=jdbc:h2:mem:racuni-db
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# naj Hibernate sam ustvari/posodobi shemo
spring.jpa.hibernate.ddl-auto=update

# prikaži SQL poizvedbe v konzoli
spring.jpa.show-sql=true

# H2 spletna konzola
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# port
server.port=8085
```

27

Dodamo entiteto **Racun**

```
package si.um.feri.rest.racuni;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Racun {
    @Id
    private String id;
    private String datum;
    private String kraj;
    private double znesek;
    private String ppld;
    private String davcnaSt;
}
```

28

Dodamo entiteto RacunRepository

```
package si.um.feri.rest.racuni;
import org.springframework.data.jpa.repository.JpaRepository;

public interface RacunRepository extends JpaRepository<Racun, String> {
}
```

JpaRepository<Racun, String> zagotovi:

- findAll(),
- findById(id),
- save(racun),
- deleteById(id)
- itd.

29

RacunController

```
package si.um.feri.rest.racuni;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.net.URI;
import java.util.List;
import java.util.UUID;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

@RestController
@RequestMapping("/api/racuni")
public class RacunController {
    private final RacunRepository racunRepository;
    public RacunController(RacunRepository racunRepository) { this.racunRepository = racunRepository; }

    @GetMapping
    public List<Racun> vsiRacuni() { // implementacija }

    @GetMapping("/{id}")
    public ResponseEntity<Racun> enRacun(@PathVariable String id) { // implementacija }

    @PostMapping
    public ResponseEntity<Racun> novRacun(@RequestBody Racun racun) { // implementacija }

    @PutMapping("/{id}")
    public ResponseEntity<Void> posodobiRacun(@PathVariable String id, @RequestBody Racun racun) { // implementacija }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> izbrisRacun(@PathVariable String id) { // implementacija }
}
```

30

RacunController - **GET**

```
// GET /api/racuni -> seznam vseh računov
@GetMapping
public ResponseEntity<List<Racun>> getAll(){
    List<Racun> vsiRacuni = racunRepository.findAll();
    if (vsiRacuni.isEmpty()){
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.ok(vsiRacuni);
}

// GET /api/racuni/{id} -> en račun
@GetMapping("/{id}")
public ResponseEntity<Racun> enRacun(@PathVariable String id) {

    Racun racun = racunRepository.findById(id).orElse(null);
    if (racun == null) {
        return ResponseEntity.notFound().build();
    }
    return ResponseEntity.ok(racun);
}
```

31

RacunController - **POST**

```
// POST /api/racuni -> nov račun
@PostMapping
public ResponseEntity<Racun> novRacun(@RequestBody Racun racun) {

    // če id ni podan, ga generiramo
    if (racun.getId() == null || racun.getId().isBlank()) {
        racun.setId(UUID.randomUUID().toString());
    }
    Racun shranjen = racunRepository.save(racun);

    // zgradimo URI do novega vira: /api/racuni/{id}
    URI location = ServletUriComponentsBuilder
        .fromCurrentRequest() // /api/racuni
        .path("/{id}") // doda /{id}
        .buildAndExpand(shranjen.getId())
        .toUri();

    return ResponseEntity
        .created(location) // nastavi status 201 + Location header
        .body(shranjen); // telo odgovora = ustvarjeni vir
}
```

32

RacunController - **PUT**

```
// PUT /api/racuni/{id} -> posodobitev obstoječega računa
@PutMapping("/{id}")
public ResponseEntity<Void> posodobiRacun( @PathVariable String id,
    @RequestBody Racun racun) {

    if (!racunRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    racun.setId(id);
    racunRepository.save(racun);
    return ResponseEntity.noContent().build();
}
```

33

RacunController - **DELETE**

```
// DELETE /api/racuni/{id} -> brisanje
@DeleteMapping("/{id}")
public ResponseEntity<Void> izbrisiRacun(@PathVariable String id) {
    if (!racunRepository.existsById(id)) {
        return ResponseEntity.notFound().build();
    }

    racunRepository.deleteById(id);
    return ResponseEntity.noContent().build();
}
```

34

Razvoj storitev REST na osnovi platforme .NET

35

ASP.NET Core

Ogrodje, ki poenostavi izgradnjo storitev HTTP



Omogoča razvoj storitev za različne odjemalce

Spletne brskalnike

Mobilne naprave

Windows aplikacije

....



Platforma za razvoj RESTful aplikacij na osnovi ogrodja .NET

36

ASP.NET Core

- Odprtokodno, večplatformsko ogrodje za razvoj sodobnih spletnih aplikacij in spletnih API-jev na ogrodju .NET
- Optimizirano za:
 - oblačna okolja (cloud-native),
 - mikrororitve,
 - moderne spletne in mobilne odjemalce
- Zasnovano na novo (ni zgrajeno neposredno na starem ASP.NET / System.Web)
- Temelji na modularnih komponentah, ki zmanjšajo količino konfiguracije in omogočajo lažje sestavljanje rešitev
- Deluje na Windows, macOS in Linux

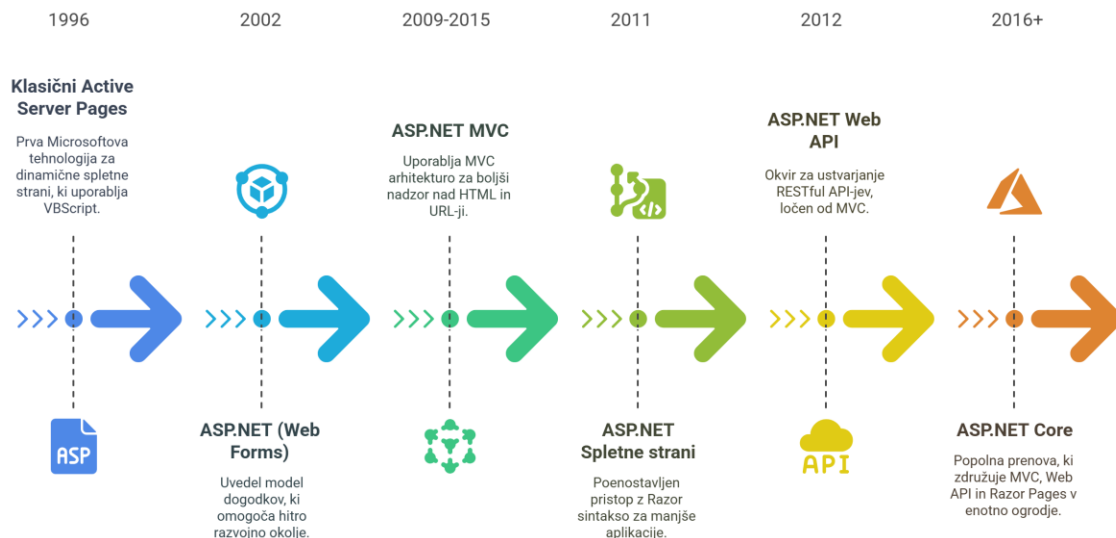
37

Zakaj ASP.NET Core?

- Prve različice ASP.NET so nastale na začetku 2000-ih na ogrodju .NET Framework
- V teh letih je bil ASP.NET široko sprejet za razvoj spletnih aplikacij
 - (Web Forms, kasneje MVC in Web API)
- Sčasoma so se pojavile nove zahteve:
 - sodobni JavaScript odjemalci (SPA),
 - oblačne arhitekture,
 - mikrororitve,
 - večplatformski razvoj
- Postalo je jasno, da klasični ASP.NET težko sledi novim arhitekturnim zahtevam
- ASP.NET Core prinaša novo arhitekturo, ki odpravi omejitve starega modela

38

Evolucija ASP in ASP.NET



39

Od ASP.NET 5 do ASP.NET Core 1.0

Pri podjetju Microsoft so želeli poenotiti ASP.NET MVC in ASP.NET Web API

Rezultat je bilo novo ogrodje, sprva poimenovano ASP.NET 5, ki je pozneje dobilo ime ASP.NET Core 1.0

Ogrodje ASP.NET Core je v veliki meri napisano na novo, združuje MVC, Razor Pages in Web API na skupnem ogrodju, in uporablja enoten nabor komponent za obdelavo HTTP zahtev

40

ASP.NET Core in System.Web / NuGet

ASP.NET Core ne uporablja več knjižnice System.Web.dll

- klasični ASP.NET je bil močno vezan na System.Web, ki je bil obsežen in težje razširljiv
- ASP.NET Core opusti to odvisnost → manjša poraba pomnilnika, boljša zmogljivost

Temelji na paketih NuGet

- vse je modularno,
- v aplikacijo vključimo le tiste knjižnice, ki jih res potrebujemo

Posledice

- manjša velikost aplikacij,
- hitrejši zagon,
- lažje posodabljanje posameznih delov brez poseganja v celotno ogrodje

41

Zakaj ASP.NET Core? – učinkovitost in zmogljivost

Večja učinkovitost in varnost

- zahteve se obdelujejo skozi verigo komponent – middleware,
- natančno lahko nadzorujemo, kaj se z zahtevo zgodi v posamezni fazi
- mehanizme za varnost (CORS, avtentikacija, avtorizacija ...) vklapljammo modularno
- asinhrona obdelava (async/await) je podprta skozi celoten sklad

Večja zmogljivost

ASP.NET Core sodi med najzmogljivejša ogrodja za spletne aplikacije v različnih primerjavah

lahko teče na Linux, Windows ali macOS,

omogoča več možnosti gostovanja in zmanjšanje stroškov infrastrukture

42

Zakaj ogrodje ASP.NET Core?

Ogrodje, zasnovano
za sodobne
aplikacije

Enoten pristop za

Pripravljeno za
oblak in
porazdeljene
sisteme

spletni
uporabniški
vmesnik (MVC,
Razor Pages),

spletne API-je
(Web API)

43

Ogrodje, zgrajeno za sodobne aplikacije

Ogrodje,
zgrajeno za
sodobne
aplikacije

Podpira razvoj

- klasičnih spletnih aplikacij (MVC, Razor Pages),
- REST API-jev in drugih HTTP storitev,
- aplikacij v realnem času (SignalR),
- mikrostoritev,
- oblačnih in cloud-native arhitektur.

Zasnovano je tako, da odpravlja ključne omejitve klasičnega ASP.NET in omogoča sodobne razvojne pristope.

44

Enoten pristop za UI in API

ASP.NET Core združuje MVC, Razor Pages, Web API na skupnem temelju

Ni več ločenih ogrodij za »spletne strani« in API-je

To omogoča:

lažje testiranje,

boljšo ponovno uporabo kode,

enostavno integracijo z modernimi ogrodji JavaScript (npr. Angular, React, Vue).

45

Ogrodje, pripravljeno za oblak

ASP.NET Core aplikacije so zasnovane za delovanje v oblačnih okoljih:

Microsoft Azure

Amazon Web Services (AWS)

Google Cloud

in drugih ponudnikih

Vključuje

vgrajen sistem za konfiguracijo (datoteke, okoljske spremenljivke, tajnosti ...)

podporo za beleženje dnevnikov (logging)

vgrajen Dependency Injection (DI)

razširljivost z vmesnim slojem (middleware) za dodatne funkcionalnosti

46

Zakaj ASP.NET Core?

***Medplatformsko,
visoko zmogljivo,
odprtokodno ogrodje
za razvoj modernih, cloud-native, spletnih
rešitev.***

47

Zakaj ASP.NET Core

Medplatformsko

deluje na Windows, macOS in Linux

Visoko zmogljivo

po primerjavah sodi med najhitrejša spletna ogrodja

Odprtokodno

razvito kot open-source projekt na GitHubu, pod okriljem .NET Foundation

48

Rešitve za oblak ali lokalno infrastrukturo



Aplikacije lahko enako dobro

gostimo lokalno (on-premises),

nameščamo na lastne strežnike,

uporabljamo na napravah na robu omrežja
(edge computing)



ASP.NET Core je zasnovano kot cloud-native ogrodje

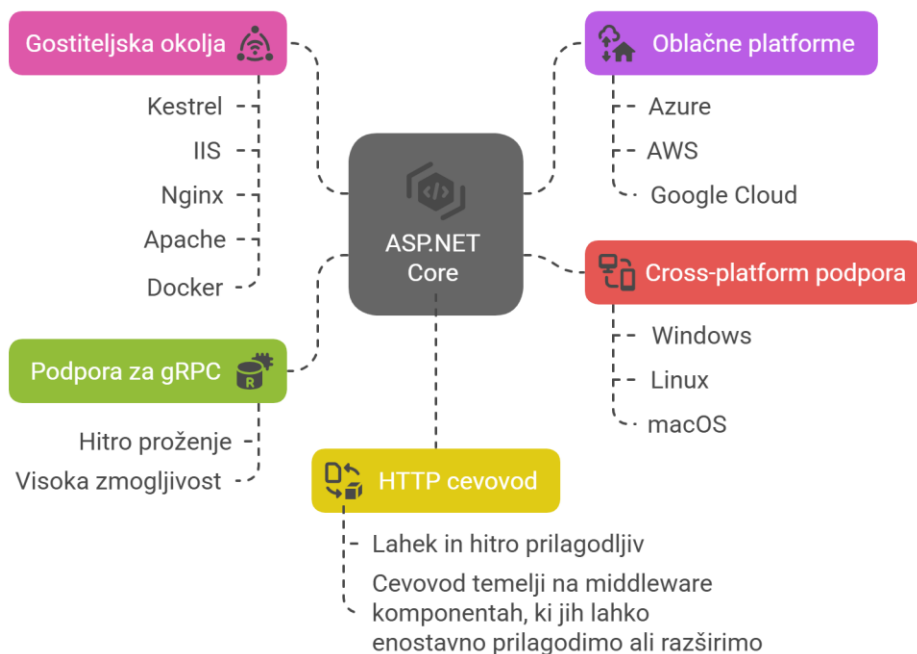
odlična podpora za

- Docker,
- Kubernetes,
- Azure App Services

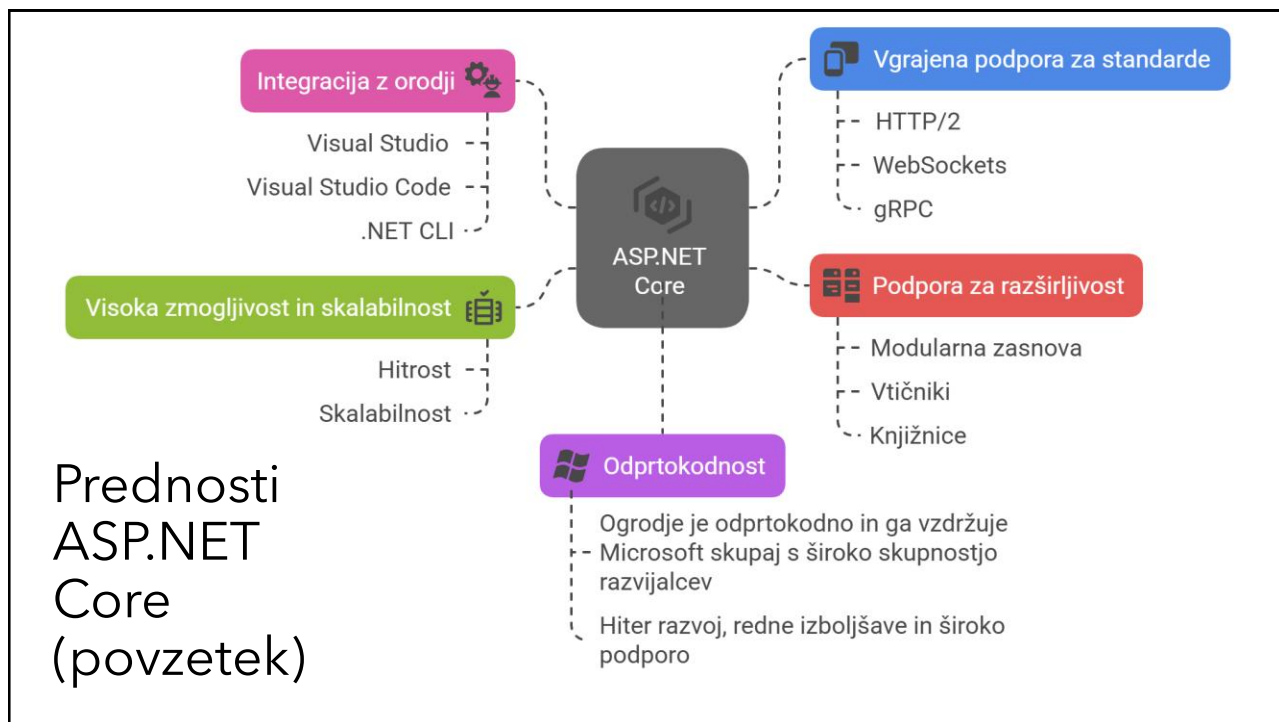
...

49

Prednosti ASP.NET Core (povzetek)



50



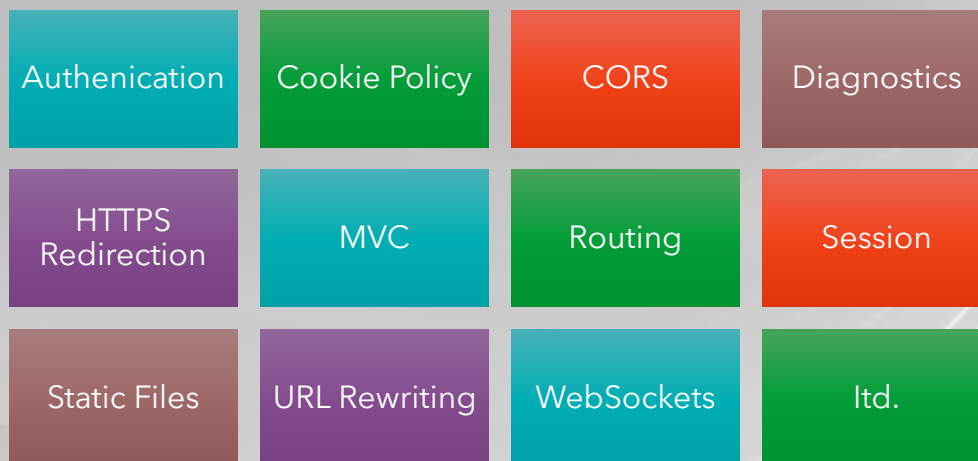
51

ASP.NET Core Web API - glavni koncepti

REST arhitektura	- Sledi principom REST (Representational State Transfer) - Poudarek na uporabi standardnih HTTP metod: GET, POST, PUT, DELETE
Vzorec MVC (Model-View-Controller)	- Uporablja MVC arhitekturo za organizacijo kode - V Web API-jih običajno brez pogleda (View) - fokus na logiki in podatkih
Brez stanja (Statelessness)	- Strežnik ne hrani informacij o stanju odjemalca med zahtevki - Vsaka zahteva vsebuje vse potrebne podatke
Arhitektura odjemalec-strežnik	- Odjemalec (mobilna aplikacija, spletna stran, druga storitev) pošilja zahteve HTTP - Strežnik vrača odgovore v obliki JSON ali XML
HTTP je osnovni protokol	- Osnova za komunikacijo je HTTP - Podpora tudi za HTTP/2 (hitrejša, boljša podpora za paralelne zahteve)
Asinhrona obdelava (async/await)	- Temelji na asinhronih operacijah - Omogoča boljšo skalabilnost in obdelavo večjega števila zahtev

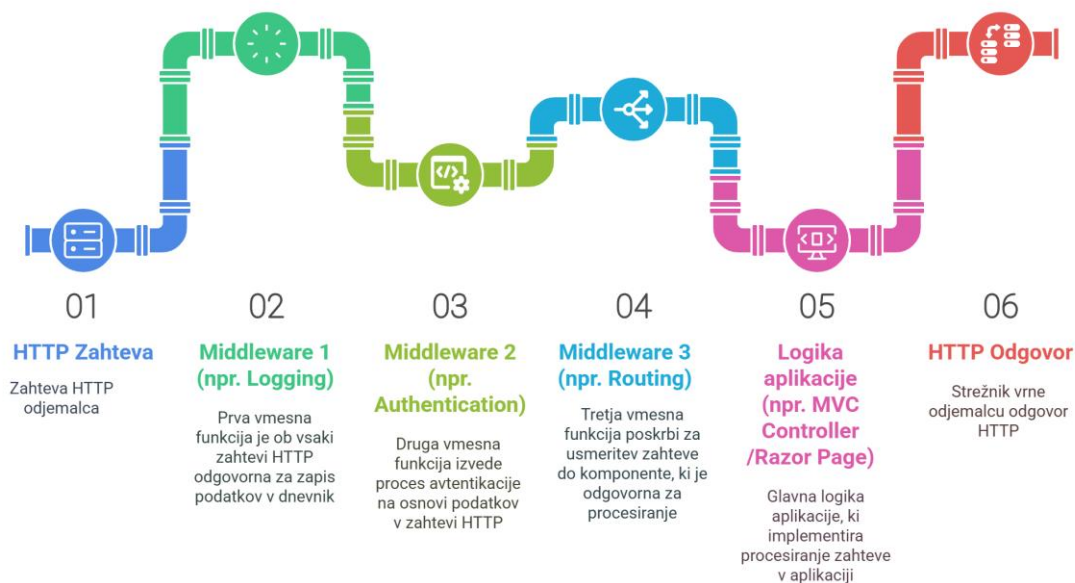
52

Funkcije na vmesnem sloju, ki jih zagotavlja ogrodje



53

Primer cevododa



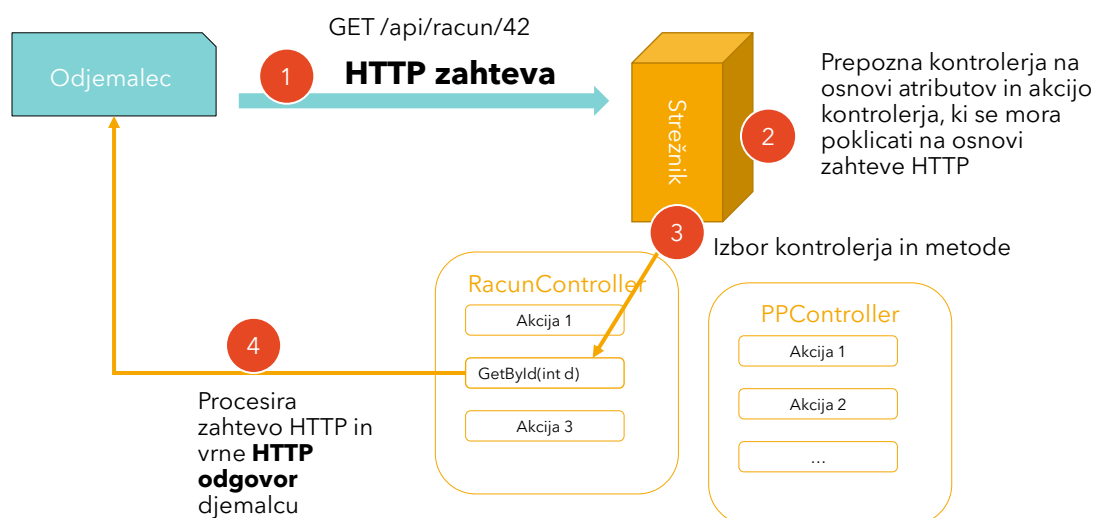
54

ASP.NET Core Web API Controller

- Ob zahtevi HTTP strežnik določi:
 - kontroler, ki ustreza konfigurirani poti ([Route], [HttpGet], ...),
 - akcijsko metodo, ki obdeluje zahtevo glede na HTTP metodo (GET, POST, ...)
- Usmerjanje (routing) temelji na atributih na razredih in metodah
 - (attribute-based routing), ne več na RouteConfig.cs
- Metode kontrolerja lahko vračajo:
 - **IActionResult** ali **IActionResult<T>** – priporočena pristopa (fleksibilnost, statusne kode),
 - **konkretni tip objekta** (Racun, IEnumerable<Racun> ...) – ASP.NET Core ga serializira v JSON (privzeto),
 - **void** – ni priporočljivo,
 - **HttpResponse** / **HttpResponseMessage** – možna, a v ASP.NET Core redko uporabljena izbira

55

ASP.NET Web API - glavni koncepti



56

ASP.NET Web API - kontrolerji in akcije

- **ActionResult** je priporočen povratni tip, ker
 - omogoča vračanje standardnih HTTP odzivov (200, 201, 400, 404, ...),
 - je bolj fleksibilen kot goli object ali void,
 - omogoča boljše testiranje in jasnejši nadzor nad odzivi
- Kontroler omogoča uporabo vgrajenih metod za vračanje standardnih rezultatov:
 - **Ok(object)** - ✔ 200 OK
 - **Created(uri, object)** - 🆕 201 Created
 - **BadRequest()** - ✖ 400 Bad Request
 - **NotFound()** - 🔍 404 Not Found
 - **Unauthorized()** - 🚫 401 Unauthorized
 - **NoContent()** - ⬜ 204 No Content
 - **ObjectResult(object)** - generičen rezultat (možnost nastavitve statusne kode)

57

Uporaba storitev in vgrajene odvisnosti (DI)

- ASP.NET Core ima vgrajeno podporo za odvisnosti (DI) v samem ogrodju
- Storitve registriramo v Program.cs (ali v prejšnjih verzijah v Startup.cs), npr.:

```
builder.Services.AddScoped<IRacunService, RacunService>();
```

- Kontroler storitev prejme preko **konstruktorja**:

```
public class RacuniController : ControllerBase
{
    private readonly IRacunService _racunService;
    public RacuniController(IRacunService racunService)
    { _racunService = racunService; }
}
```

58

Življenjski cikel storitev

Tip	Opis	Primer uporabe
Singleton	ena instanca v celotni življenjski dobi aplikacije	zapisovalnik dnevnikov (logger), globalni cache, konfiguracijske storitve.
Scoped	nova instanca za vsako HTTP zahtevo	storitve, ki delajo z trenutno zahtevo, kot so storitve nad podatkovno bazo (DbContext).
Transient	vsakič, ko je storitev potrebna, se ustvari nova instanca	lahke, kratkožive komponente brez stanja

59

Uporaba Swagger za dokumentacijo API-ja

- ASP.NET Core omogoča zelo enostavno integracijo Swagger / OpenAPI:
 - samodejno generiranje opisa API-ja,
 - interaktivni uporabniški vmesnik za testiranje (Swagger UI).
- Osnovna konfiguracija v Program.cs:

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
var app = builder.Build();
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

60

Primer WebAPIja Racun na osnovi ASP.NET Core

- Storitve z naslednjim vmesnikom:

API	Opis	Telo zahteve	Telo odgovora
GET /api/racuni	Vrne seznam vseh računov	/	polje objektov racun (XML ali JSON)
GET /api/racuni/{id}	Vrne račun z vrednostjo id	/	objekt Racun (XML ali JSON)
POST /api/racuni	Ustvari nov vir račun	objekt Racun	objekt Racun (XML ali JSON)
PUT /api/racuni/{id}	Posodobi obstoječ vir račun z vrednostjo ISBN	objekt Racun	/
DELETE /api/racuni/{id}	Zbriše vir račun z vrednostjo ISBN	/	/

61

Ustvarimo projekt

```
dotnet new webapi -n.AspNetCoreRacuni --use-controllers  
cd.AspNetCoreRacuni  
code .
```

62

Model Racun (v mapi /Models)

```
namespace AspNetCoreRacuni.Models
{
    public class Racun
    {
        public string? Id { get; set; }
        public string? Datum { get; set; }
        public string? Kraj { get; set; }
        public double Znesek { get; set; }
        public string? PpId { get; set; }
        public string? DavcnaSt { get; set; }
    }
}
```

63

Dodamo **RacuniController.cs** (v mapo Controllers)

```
using System.Collections.Concurrent;
using AspNetCoreRacuni.Models;
using Microsoft.AspNetCore.Mvc;

namespace AspNetCoreRacuni.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class RacuniController : ControllerBase
    {
        // preprost "in-memory" pomnilnik: id -> račun
        private static readonly ConcurrentDictionary<string, Racun> _data =
            new ConcurrentDictionary<string, Racun>();

        // + metode za implementacijo GET, POST, PUT, DELETE

    }
}
```

64

RacunController - **GET**

```
// GET /api/racuni -> vsi računi
[HttpGet]
public ActionResult<IEnumerable<Racun>> GetVsi()
{
    return Ok(_data.Values);
}

// GET /api/racuni/{id} -> en račun
[HttpGet("{id}")]
public ActionResult<Racun> GetEn(string id)
{
    if (!_data.TryGetValue(id, out var racun))
    {
        return NotFound();
    }

    return Ok(racun);
}.
```

65

RacunController - **POST**

```
// POST /api/racuni -> nov račun
[HttpPost]
public ActionResult<Racun> Post([FromBody] Racun racun)
{
    // če Id ni podan, ga generiramo
    if (string.IsNullOrEmpty(racun.Id))
    {
        racun.Id = Guid.NewGuid().ToString();
    }

    // če že obstaja, ga prepišemo
    _data[racun.Id] = racun;

    // 201 Created + Location header na GET /api/racuni/{id}
    return CreatedAtAction(
        nameof(GetEn), // ime metode za branje enega računa
        new { id = racun.Id }, // route values
        racun // telo odgovora
    );
}
```

66

RacunController - **PUT**

```
// PUT /api/racuni/{id} -> posodobitev
[HttpPut("{id}")]
public IActionResult Put(string id, [FromBody] Racun racun)
{
    if (!_data.ContainsKey(id))
    {
        return NotFound();
    }

    racun.Id = id;
    _data[id] = racun;

    return NoContent();
}
```

67

RacunController - **DELETE**

```
// DELETE /api/racuni/{id} -> brisanje
[HttpDelete("{id}")]
public IActionResult Delete(string id)
{
    if (!_data.ContainsKey(id))
    {
        return NotFound();
    }

    _data.TryRemove(id, out _);
    return NoContent();
}
```

68

REST storitev na osnovi ASP.NET Core minimal API

69



ASP.NET Core Minimal API

Arhitekturno ogrodje za razvoj HTTP APIjev, ki zahtevajo minimalno vključevanje odvisnih knjižnic

Primerno predvsem pri razvoju mikrostoritev in aplikacij, kjer se zahteva vključevanje minimalnega števila datotek, lastnosti in odvisnosti v ASP.NET Core

70

Kaj je Minimal API?

= poenostavljena alternativa Controller/Action pristopu

Vse se definira z MapGet, MapPost, ...

Primeren za majhne API-je, mikrostoritve, prototipe

Na voljo od .NET 6 naprej

71

Primer osnovnega API-ja

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
```

```
app.MapGet("/hello", () => "Pozdravljen, svet!");
app.Run();
```

Brez kontrolerjev, modelov, routing atributov

Hitro in berljivo

72

ASP.NET Core Minimal API : Express (Node.js)

ASP.NET Core Minimal API

Ni potrebe po kontrolerjih in razredih
 MapGet, MapPost, ...
 Handler funkcije so pogosto inline
 Hitro postavljanje preprostih endpointov



Express (Node.js)

Ni potrebe po razredih, le funkcije/handlerji
 app.get, app.post, ...
 Handler funkcije so tudi v Express inline
 Hitro postavljanje preprostih endpointov

73

ASP.NET Core API vs Minimal API

Vidik	Klasičen API	Minimal API
Routing	[HttpGet], [HttpPost], ...	app.MapGet(...), app.MapPost(...)
Arhitektura	Controller + atributi	Funkcionalna z metodami Map
Zmožnost testiranja	Visoka (ločeni razredi)	Nižja (potrebno prilagoditi, ročno)
DI (odvisnosti)	Konstruktorji v kontrolerju	Preko parametrov lambda funkcij
Validacija modela	ModelState.IsValid + [Required]	Ročna validacija (ali z FluentValidation)

74

Kdaj uporabiti Minimal API?

Uporabimo, ko:

- gradimo preprost API ali prototip
- razvijamo mikrosoritev
- želimo kratko kodo
- želimo hitro namestiti funkcionalnost

Premislimo, če:

- API postaja kompleksen
- potrebujemo podporo za napredno validacijo, filtre, middleware

75

Ustvarimo nov projekt

dotnet new web -n MinimalApiRacuni

76

Entitetni razred Racun

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace MinimalApiRacuni.Models
{
    public class Racun
    {
        public string? Id { get; set; }
        public string? Datum { get; set; }
        public string? Kraj { get; set; }
        public double Znesek { get; set; }
        public string? PpId { get; set; }
        public string? DavcnaSt { get; set; }
    }
}
```

77

IRacunRepository

```
namespace MinimalApiRacuni.Models
{
    public interface IRacunRepository
    {
        Racun Get(string id);
        List<Racun> GetAll();
        Racun Add(Racun racun);
        void Update(string id, Racun racun);
        void Delete(string id);
    }
}
```

78

```

using System.Collections.Concurrent;
namespace MinimalApiRacuni.Models
{
    public class RacunRepository : IRacunRepository
    {
        private readonly ConcurrentDictionary<string, Racun> racuni = new ConcurrentDictionary<string, Racun>();
        public RacunRepository()
        {
            if (this.racuni.Count == 0)
            {
                //dodaj nekaj testnih računov
            }
        }
        public Racun Get(string id)
        {
            racuni.TryGetValue(id, out var racun);
            return racun;
        }
        public List<Racun> GetAll()
        {
            return racuni.Values.ToList();
        }
        public Racun Add(Racun racun)
        {
            racun.Id = Guid.NewGuid().ToString();
            if (!racuni.TryAdd(racun.Id, racun))
            {
                throw new InvalidOperationException("Dodajanje računa ni uspelo.");
            }
            return racun;
        }
        public void Update(string id, Racun racun)
        {
            if (!racuni.ContainsKey(id))
            {
                throw new KeyNotFoundException($"Računa z ID {id} nisem našel.");
            }
            racun.Id = id;
            racuni[id] = racun;
        }
        public void Delete(string id)
        {
            if (!racuni.TryRemove(id, out _))
            {
                throw new KeyNotFoundException($"Računa z ID {id} nisem našel.");
            }
        }
    }
}

```

RacunRepository

79

Program.cs

```

using MinimalApiRacuni.Models;
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

IRacunRepository racunRepository = new RacunRepository();

app.UseHttpsRedirection();

// implementacija GET, POST, PUT in DELETE

app.Run();

```

80

GET

```
app.MapGet("/api/racuni", () =>
{
    var racuni = racunRepository.GetAll();
    return Results.Ok(racuni);
});

app.MapGet("/api/racuni/{id}", (string id) =>
{
    var racun = racunRepository.Get(id);
    if (racun == null)
    {
        return Results.NotFound("Račun z id= " + id + " ni bil najden");
    }
    return Results.Ok(racun);
});
```

81

POST

```
app.MapPost("/api/racuni", (Racun racun, HttpContext context) =>
{
    // validacija (ker v Minimal API ni samodejne ModelState validacije)
    if (string.IsNullOrEmpty(racun.Datum) ||
        string.IsNullOrEmpty(racun.Kraj) ||
        string.IsNullOrEmpty(racun.PpId) ||
        string.IsNullOrEmpty(racun.DavcnaSt) ||
        racun.Znesek <= 0)
    {
        return Results.BadRequest("Podatki niso veljavni.");
    }

    var nov = racunRepository.Add(racun);
    return Results.Created($"{context.Request.Scheme}://{context.Request.Host}
        /api/racuni/{nov.Id}", nov);
});
```

82

PUT

```
app.MapPut("/api/racuni/{id}", (string id, Racun racun) =>
{
    var najdi = racunRepository.Get(id);
    if (najdi is null)
    {
        return Results.NotFound($"Račun z ID={id} ne obstaja.");
    }

    racunRepository.Update(id, racun);
    return Results.NoContent();
});
```

83

DELETE

```
app.MapDelete("/api/racuni/{id}", (string id) =>
{
    var najdi = racunRepository.Get(id);
    if (najdi is null)
    {
        return Results.NotFound($"Račun z ID={id} ne obstaja.");
    }

    racunRepository.Delete(id);
    return Results.NoContent();
});
```

84

Razvoj storitev REST na osnovi JavaScript

85

Primer RESTFull storitve na osnovi Node.js

- Storitev z naslednjim vmesnikom:

API	Opis	Telo zahteve	Telo odgovora
GET /api/racuni	Vrne seznam vseh računov	/	polje objektov racun (XML ali JSON)
GET /api/racuni/{id}	Vrne račun z vrednostjo id	/	objekt Racun (XML ali JSON)
POST /api/racuni	Ustvari nov vir račun	objekt Racun	objekt Racun (XML ali JSON)
PUT /api/racuni/{id}	Posodobi obstoječ vir račun z vrednostjo ISBN	objekt Racun	/
DELETE /api/racuni/{id}	Zbriše vir račun z vrednostjo ISBN	/	/

86

Primer storitve RESTful storitve

- Osnova:
- Express
- body-parser
- Mongoose

87

body-parser

- Omogoča razpoznavanje podatkov iz telesa HTTP zahteve na vmesnem/povezovalnem nivoju preden podatke dobijo kontrolniki
- API, ki zagotavlja naslednje razpoznavalnike:
 - JSON body parser
 - Omogoča razpoznavo samo json, podpira samo Unicode
 - Raw body parser
 - Celotno telo razpozna kot Buffer
 - Text body parser
 - Podatke razpozna kot string
 - URL-encoded form body parser
 - Razpozna vsebino body, kjer so podatki url-encoded

88

body-parser

- Ne omogoča razpoznavo multi-part sporočil
- Za učinkovito procesiranje kompleksnih vsebin zahtev so na voljo dodatni moduli
 - busboy
 - multiparty
 - formidable
 - multer
 - ...

89

Vzpostavitev ogrodja APIja

```
{
  "name": "racunirestapi",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
  },
  "author": "",
  "license": "ISC",
  "dependencies": {
    "nodemon": "^2.0.2",
    "express": "^4.17.1",
    "mongoose": "^5.9.4",
    "cors": "^2.8.5"
  }
}
```

90

Model (Racun)

- */models/***racun.js**:

```
const mongoose = require('mongoose');
const racunSchema = new mongoose.Schema({
  id: {
    type: String,
    required: true
  },
  datum: {
    type: String,
    required: true
  },
  kraj: {
    type: String,
    required: true
  },
  znesek: {
    type: Number,
    required: true
  },
  ppId: {
    type: String,
    required: true
  },
  davcnaSt: {
    type: String,
    required: true
  }
});
const Racun = mongoose.model('Racun', racunSchema);
module.exports = Racun;
```

91

Inicializacija storitve (index.js)

```
const express = require('express');
const mongoose = require('mongoose');
const { MongoMemoryServer } = require("mongodb-memory-server");
const racuniRoutes = require('./routes/racuni');
const cors = require('cors');

const PORT = process.env.PORT || 3000;

async function connectInMemoryMongo() {
  const mongod = await MongoMemoryServer.create();
  const uri = mongod.getUri();
  await mongoose.connect(uri);
  console.log("In-memory MongoDB connected:", uri);
  return mongod;
}

connectInMemoryMongo().catch(console.error);

const app = express();
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use('/api/racuni', racuniRoutes);
app.use(cors());
app.listen(PORT, () => {
  console.log('API za račune je na posluša na http://localhost:${PORT}/api/racuni');
});
```

92

REST API vmesnik (/routes/racuni.js)

```
const Racun = require('../models/racun');
const express = require('express');
const router = express.Router();
```

+ implementacija metod

```
GET
POST
PUT
DELETE
```

```
module.exports = router;
```

93

GET /api/racun/:id

```
router.get('/', async (req, res)=>{
  try {
    let racuni = await Racun.find({});
    res.json(racuni);
  } catch (error) {
    return res.status(500).json({ msg: "Napaka na strežniku" });
  }
});

router.get('/:id', async (req, res)=>{
  try {
    let racun = await Racun.findById(req.params.id);
    if (!racun){
      res.status(404).json({msg: `Račun z id=${req.params.id} ni bil najden`});
    }
    res.json(racun);
  } catch (error) {
    return res.status(500).json({ msg: "Napaka na strežniku" });
  }
});
```

94

POST /api/racun

```
router.post('/', async(req, res)=>{
  try {
    const racun = new Racun(req.body);
    await racun.save();
    let racunUri =
      `${req.protocol}://${req.headers.host}${req.originalUrl}/${racun._id}`;

    res.setHeader('Location', racunUri);
    res.json(racun);
  } catch (err) {
    res.status(500).send(err);
  }
});
```

95

PUT /api/racun/:id

```
router.route('/:id').put(async(req, res) => {
  if (req.params.id !== req.body._id) {
    res.status(400).json('IDja nista enaka!')
  } else {
    try {
      const racun = await
        Racun.findByIdAndUpdate(req.params.id, req.body);
      await racun.save();
      res.json({ msg: 'Račun je bil posodobljen' });
    } catch (err) {
      res.status(500).send(err);
    }
  }
});
```

96

DELETE /api/racun/:id

```
router.delete("/:id", async (req, res)=>{  
  
  try {  
    const brisi = await  
    Racun.findByIdAndDelete(req.params.id);  
  
    if (!brisi) {  
      res.status(404).json({ msg: `Račun z id=${req.params.id} ne obstaja` });  
    } else {  
      res.status(200).json({ msg: 'Račun je bil zbrisan' });  
    }  
  } catch (err) {  
    res.status(500).send(err)  
  }  
  
});
```

97

Tehnologije in standardi za opis storitev

98

Tehnologije/standardi za opis storitev

Storitveno orientirane arhitekture

Tehnologija	Predstavitev	Standardizacija
WS-*	XML	Različni
UDDI	XML	OASIS
SoaML	UML	OMG
UPMS	UML	/
SOA-RM	Neformalni zapis	OASIS
SOA Ontology	OWL	Open Group
WADL	XML	W3C
Core Ontology of Web Services	OWL	/

99

Tehnologije/standardi za opis storitev

Semantične spletne storitve

Tehnologija	Namen	Predstavitev	Standardizacija
OWL-S	Avtomatizacija	OWL	/
WSMO	Avtomatizacija	WSML	/
SAWSDL	Izmenjava	XML	W3C
SA-REST	Izmenjava	XML	W3C
RO-SOA	Naslavljanje	RDFS	OASIS
WSMO-Lite	Izmenjava	XML	W3C
MicroWSMO	Izmenjava	RDFS	/
Minimal Service Model	Izmenjava	RDFS	/

100

WADL

- **Web Application Discovery Language**
- Slovar za opis spletnih aplikacij
- Ustvarjen 2005 (Marc Hadley, Sun Microsystems)
 - Nastal je zaradi **pomanjkljivosti jezika WSDL** na področju opisovanja in razvoja RESTful spletnih storitev
 - 31.8.2009 je bil poslan na W3C (status: Membership Submission) (<http://www.w3.org/Submission/wadl/>)
- WADL je WSDL za RESTful spletne storitve
 - Strojno-berljiv opis storitev/aplikacij, ki temeljijo na HTTP protokolu
- Format XML
 - Neodvisen od programskega jezika, platforme

101

Struktura dokumenta WADL

```

<application>
  <doc>*
  <grammars/> ?
  <resources base="anyURI"?>
    <doc>*
    <resource path="template" type="anyURI" +?>+
      <doc>*
      <param/>*
      (<method/> | <resource/>)+
    </method>
  </resource>
</resources>
(<method/> | <representation/> | <fault/> |
  <resource_type/>)*
</application>

```

* → 0 ali več
 ? → 0 ali 1
 + → 1 ali več

102

Struktura opisa metode

```
<method name="NMTOKEN" id="ID" href="anyURI" +?>
  <doc>*
  <request> ?
    <param/>*
    <representation/>
  </request>
  <response>?
    (<representation/> | <fault/>)*
  </response>
</method>
```

103

Orodja za WADL

- Odprtokodni projekt - <http://wadl.java.net/>
- Generira razrede za izgradnjo odjemalca
- Orodje, ki ga lahko zaženemo v ukazni vrstici ali kot Ant opravilo
- Uporablja JAXB za XML procesiranje
- WADL-first razvoj
 - **wadl2java**
 - <http://cxf.apache.org/docs/jaxrs-services-description.html#JAXRSServicesDescription-WADLfirstDevelopment>

104

RESTful Service Description Language (RSDL)

- **RESTful Service Description Language (RSDL)**
- XML besednjak za načrtovanje in dokumentiranje RESTful storitev
- Uporablja pristop k načrtovanju REST, ki
 - temelji na hipermedijskih dokumentih,
 - zahteva, da ima storitev eno vstopno točko,
 - usmerjen na načrtovanje virov, povezav in tipov medijskih vsebin

105

RSDL : WADL

- Jezik WADL je kritiziran ker dejansko vzpoduja **izgradnjo tesno sklopljene arhitekture na osnovi URI** med odjemalcem in strežnikom
 - WADL direktno izpostavi URI in fiksne poti - ne temelji na povezavah, dokumentih
 - definira specifične napake namesto da bi uporabljal HTTP protokol in zmožnosti protokola
 - ...
- Iz vidika REST je glavni problem WADL-a izpostavljanje vmesnikov preko **statičnih metapodatkov**
 - namesto spodbujanja/opisovanja pravil kako poiskati in uporabiti povezave znotraj vsebine multimedije, ki jo pridobimo na osnovi zahteve - **HATEOAS**

106

Stuktura RSDL

```
<?xml version="1.0" encoding="utf-8"?>
<service name="Documents"
  identity-provider-ref="idp"
  xmlns="http://identifiers.emc.com/rsdl"
  xmlns:html="http://www.w3.org/1999/xhtml/">
```

```
  <documentation>This RESTful service provides a simple ATOM feed that allows documents to be read,
```

```
  <start ref="res-home"/>
```

```
  <media-types>
```

```
  <resources>
```

```
  <link-relations>
```

```
  <headers>
```

```
  <authentication>
```

```
  <status-codes>
```


107

Več o RSDL...

- <http://www.balisage.net/Proceedings/vol10/html/Robie01/BalisageVol10-Robie01.html>
- <https://github.com/jonathanrobie/restful-service-description-language/blob/master/examples/homedoc-atom.rsdl>

Robie, Jonathan, Rob Cavicchio, Rémon Sinnema and Erik Wilde. "RESTful Service Description Language (RSDL): Describing RESTful Services Without Tight Coupling." Presented at Balisage: The Markup Conference 2013, Montréal, Canada, August 6 - 9, 2013. In Proceedings of Balisage: The Markup Conference 2013. Balisage Series on Markup Technologies, vol. 10 (2013). doi:10.4242/BalisageVol10.Robie01.

108

JSDL

- **JSON Service Description Language**

- Jezik za definicijo vmesnika v formatu JSON
- Omogoča opis funkcionalnosti, ki jih zagotavlja HTTP spletna storitev
- Za opis tipov uporablja JSON Schema + vpeljuje dodatne strukture za definicijo JSON spletnih storitev

109

JSDL - primer

```
{
  "operations": {
    "sestej": {
      "transport": "post",
      "target": "service/sestej",
      "inputType": "json",
      "parameters": [
        {
          "type": "number",
          "name": "a"
        },
        {
          "type": "number",
          "name": "b"
        }
      ],
      "returns": {
        "type": "number"
      },
      "description": "Sesteje dve vrednosti"
    },
    "vrniOsebo": {
      "transport": "post",
      "target": "service/odstaj",
      "inputType": "json",
      "parameters": [
        {
          "type": "number",
          "name": "id"
        }
      ],
      "returns": {
        "type": "Oseba"
      }
    }
  },
  "types": [
    {
      "type": "object",
      "name": "Oseba",
      "properties": {
        "ime": {
          "type": "string"
        },
        "priimek": {
          "type": "string"
        },
        "starost": {
          "type": "number"
        }
      }
    }
  ]
}
```

110

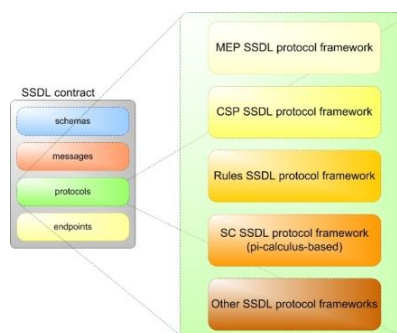
SSDL

- SOAP Service Description Language (SSDL)
- Jezik za opis spletnih storitev, ki temeljijo na SOAP
- Nastal z namenom raziskovanja prednosti/slabosti tehnologij opisovanja storitev →
 - raziskovanje prednosti sporočilno orientiranega pristopa k načrtovanju in izgradnji spletnih storitev (v primerjavi s stilom RPC)
- Eden izmed ciljev jezika SSDL je bilo tudi formalizacija modelov za opis sporočilnih interakcij

111

SSDL – glavne komponente

- **MEP SSDL protocol framework** – množica najbolj pogostih vzorcev izmenjave sporočil
<http://savas.me/ssdl/docs/v1.3/html/MEP%20SSDL%20Protocol%20Framework%20v1.3.html>
- **CSP SSDL protocol framework** – (Communicating Sequential Processes) semantika za procese pri izmenjavi večjega števila sporočil
<http://savas.me/ssdl/docs/v1.3/html/CSP%20SSDL%20Protocol%20Framework%20v1.3.html>
- **Rules SSDL protocol framework** – Definira pogoje za izmenjavo sporočil
<http://savas.me/ssdl/docs/v1.3/html/Rules%20SSDL%20Protocol%20Framework%20v1.3.html>
- **SC SSDL protocol framework** – Ogrodje za opis protokola izmenjave
<http://savas.me/ssdl/docs/v1.3/html/SC%20SSDL%20Protocol%20Framework%20v1.3.html>



112

Prihodnost SSDL?

- Uporabljal se je predvsem v akademski sferi – za potrebe raziskovanja
- Ni se prenesel v industrijske projekte
- Več: <http://savas.me/ssdl/>
- **Soya** - <http://soya.sourceforge.net/> - odprtokodna implementacija SSDL (nazadnje posodobljeno 2007)
 - Programski model in izvajalno okolje za razvoj spletnih storitev na osnovi SSDL

113

USDL

- <http://www.w3.org/2005/Incubator/usdl/>
- **Universal Service Description Language**
- Platformsko neodvisen jezik za opis storitev
- Omogoča opisovanje **RAZLIČNIH VRST STORITEV!**
- **Holističen pristop k opisovanju storitev**, ki vključuje 3 vidike:
 - tehnični vidiki
 - poslovni vidiki (lastništvo, zaračunavanje storitev, pravni vidiki...)
 - operativni vidiki

114

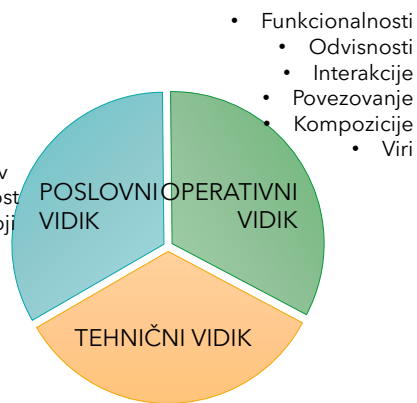
Pomen USDL

WSDL / WS-*
Ustvarjeno za računalnike (SOA)



- Ponudniki storitev in druge vloge
- Ceniki
- Nivoji storitev
- Razpoložljivost
- Splošni pogoji

USDL
Ustvarjeno za ljudi (IoS)



- Samo za IT storitve:
- Tehnični vmesnik
 - Protokoli
 - Povezave do WS-* dokumentov

115

Storitev v sklopu USDL

Aktivnost, korist ali zadovoljitev, ki je naprodaj ali je ponujena v povezavi s prodajo dobrin.

(Ameriško združenje za trženje)

116

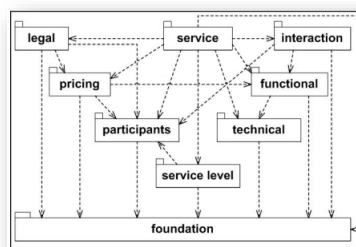
Lastnosti jezika USDL

- Predstavlja normativno in uravnoteženo poenotenje informacij o storitvi
- Dopušča za razširitve
- Strojno berljiv
- Platformsko neodvisen
- Standardiziran (W3C)

117

Shema jezika

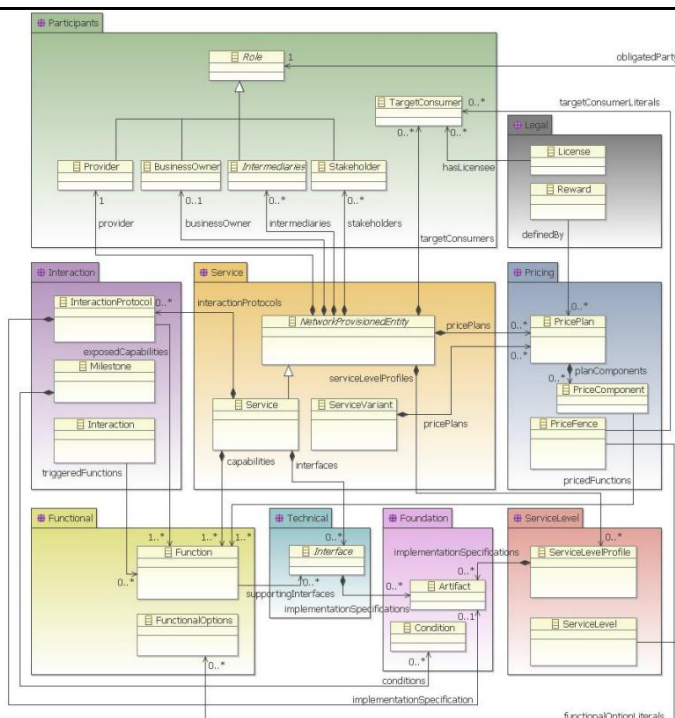
- Jezik USDL je razdeljen v več paketov oz. modulov
 - **Foundation** (koncepti, neodvisni od storitve - organizacija, oseba, vir, čas, lokacija, itd...)
 - **Service** (osnovni koncepti storitve)
 - **Service level** (kakovost storitev, zanesljivost, itd.)
 - **Technical** (tehnični vmesnik)
 - **Functional** (zmožnosti storitve)
 - **Interaction** (poslovni protokoli, obnašanje)
 - **Participants** (vloge, kot je Ponudnik, Odjemalec...)
 - **Pricing** (modeli zaračunavanja, popusti, itd.)
 - **Legal** (pravne podlage storitve)



118

Shema jezika

Povezava med moduli



119

Modul Service

- Povezuje vse vidike opisa storitve, ki so porazdeljeni po drugih modulih
- Vključuje koncepte, ki predstavljajo storitev, na primer:
 - ime storitve in verzija,
 - narava storitve,
 - ključne besede,
 - čas objave storitve,
 - povezava do dodatne dokumentacije

120



```
<ServiceElements xmi:id="Service_9505547" ExposedResource="Resource_9560387">
  <Descriptions>
    <ElementDescription xmi:id="ElementDescription_25547724">
      <Keywords>
        <KeywordsElement>pica</KeywordsElement>
        <KeywordsElement>postrežba</KeywordsElement>
        <KeywordsElement>dostava</KeywordsElement>
        <KeywordsElement>restavracija</KeywordsElement>
        <KeywordsElement>Silver</KeywordsElement>
      </Keywords>
    </ElementDescription>
  </Descriptions>
  <Guid>silver-postrežba-20110701</Guid>
  <Version>1.0</Version>
  <Name>Postrežba pice</Name>
  <AdditionalDocumentation>
    <Artifact xmi:id="Artifact_9755974">
      <Type>Mixed</Type>
      <MimeType>text/html</MimeType>
      <Uri>http://www.restavracija-silver.si</Uri>
    </Artifact>
  </AdditionalDocumentation>
  <PublicationTime xsi:type="foundation:AbsolutePointInTime" xmi:id="AbsolutePointInTime_27949740">
    <TimeZone>Europe/Ljubljana</TimeZone>
    <Value>2011-07-01T11:00:00.000+0200</Value>
  </PublicationTime>
  <Nature>Human</Nature>
</ServiceElements>
```

Prihodnost USDL?

- V prihodnosti bomo potrebovali učinkovit in razširjen standard za opis storitev
 - torej jezik, ki se ne bo ukvarjal samo s tehničnimi podrobnostmi!
 - storitve v oblakih!
- Jezik USDL je imel potencial
- Razvoj so sicer podpirali vplivni akterji (SAP, ...)
- A unified description language for human to automated services
<http://www.sciencedirect.com/science/article/pii/S0306437912000968>

123

WSDL 2.0

124

Razvoj WSDL 2.0

- WSDL 1.1 dejansko nikoli ni bil standard, je pa bil vseskozi industrijsko dobro sprejeta in podprta tehnologija
 - <http://www.w3.org/TR/wsdl>
- Zakaj?
 - Ker ni bilo drugega načina za zapis vmesnika storitev
 - Ker smo/so bili po večini zadovoljni s tem, kar omogoča
- Veliko časa in truda se je vložilo tudi v razvoj WSDL 2.0
 - **WSDL 2.0**, priporočilo, 26.06.2007
<http://www.w3.org/TR/wsdl20/>

125

WSDL 2.0 (izboljšan WSDL 1.0)

- Bolj preprosta struktura
- Polna podpora tako za SOAP kot RESTful spletne storitve
- Odstranjen element <message>.
 - Sporočila se sedaj definirajo s pomočjo XML shem
- Možnost definiranja lastnosti QoS
- Večji nabor vzorcev izmenjave sporočil (MEPs)
- Nekateri WS stroji že podpirajo in nudijo orodja za avtomatizacijo
 - Axis2 in wsdl2Java
- Sicer pa je zaenkrat v orodjih še slaba podpora oz. **podpore SPLOH NI!**

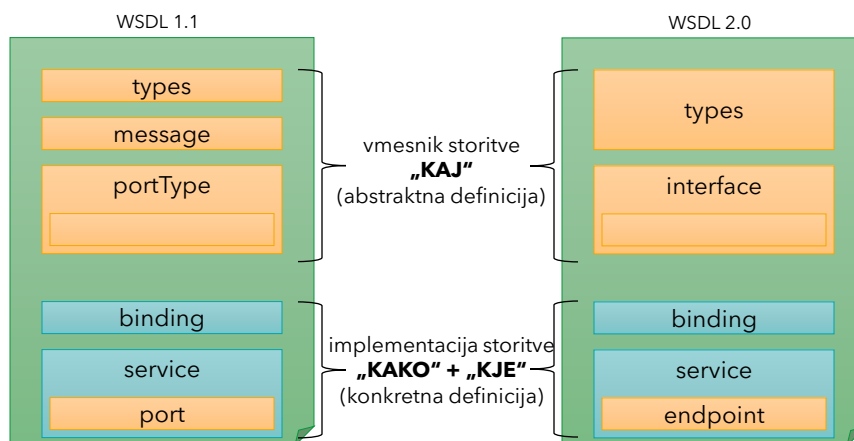
126

WSDL 1.1 → WSDL 2.0

- WSDL 2.0 integrira element `<message>` v elementu `<types>`
- Določeni elementi so se preimenovali:
 - `<definitions>` → **<description>** (korenski element)
 - `<portType>` → **<interface>**
 - `<port>` → **<endpoint>**

127

WSDL 1.1 vs WSDL 2.0



128

Element <interface>

- Abstraktna definicija storitve – brez podrobnosti o tem, kako in kje je storitev zagotovljena
- <interface> definira množico operacij in možnih napak (<fault>)
 - Element <fault> znotraj vmesnika definira možne napake, ki se lahko vrnejo odjemalcu

```
<wsdl:interface name="RadarServis">
```

```
    seznam operacij
```

```
</wsdl:interface>
```

129

Elementa <service> in <endpoint>

- Element **service** pove, kje je storitev dostopna
 - Atribut **interface** pove, kateri abstraktni vmesnik storitev implementira
 - Element service lahko vsebuje eno ali več končnih točk (endpoint)
 - Element **endpoint** definira končno točko storitve (Service Access Point), preko katere je storitev dostopna. Določa ime, binding in naslov (URL)

```
<wsdl:service name="KanglerServis" interface="tns:RadarServis">
```

```
    <wsdl:endpoint name="RadarServisSoap" binding="tns:RadarBinding"
        address="http://mom.si/radar"/>
```

```
</wsdl:service>
```

130

WSDL 2.0: 8 MEPs

- MEP (Message Exchange Pattern) = pogodba med odjemalcem in strežnikom – definira pravila in/ali način izmenjave sporočil/dokumentov
- WSDL 2.0 podpira najpogostejše oblike komunikacije
 - **In-Only**
 - **Robust In-Only**
 - **In-Out**
 - **In Optional-Out**
 - **Out-Only**
 - **Robust Out-Only**
 - **Out-In**
 - **Out-Optional-In**